SE4060 – Parallel Computing

Lab Sheet 6 – MPI – IT24103581

Exercise Answers (SE3082-Lab8)

Exercise 0 – In-class MPI activities

Program	What it demonstrates	Observed result
HelloMPI.cpp	MPI_Init, rank and processor name; rank 0 prints a different message	Rank 0 prints “Hello World!”, ranks 1–3 print “Just a normal process”
message1.cpp	Blocking MPI_Ssend / MPI_Recv of an array of 10 ints from rank 1 to rank 3	Rank 3 receives 0 10 20 … 90
message1_integer.cpp	Same, sending a single integer	Rank 3 prints y = 30
message2.cpp	Rank 1 sends 10 ints; the other ranks print their identity	Rank 3 prints 0 10 … 90; lines from other ranks interleave in a non-deterministic order

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano HelloMPI.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ HelloMPI.cpp -o hello
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./hello
Just a normal process From rank 3 machine DESKTOP-06D7UBO
Just a normal process From rank 1 machine DESKTOP-06D7UBO
Just a normal process From rank 2 machine DESKTOP-06D7UBO
Hello World! From rank 0 machine DESKTOP-06D7UBO
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 0.1 – HelloMPI on 4 processes

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano message1.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ message1.cpp -o message1
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./message1
Sending message from rank 1 to rank 3
Values received by rank 3:
0 10 20 30 40 50 60 70 80 90
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 0.2 – message1 (array send from rank 1 to rank 3)

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano message1_integer.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ message1_integer.cpp -o message1_integer
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./message1_integer
Just a normal process from rank 0 machine DESKTOP-06D7UBO
Sending message to computer 3 from computer 1
Just a normal process from rank 2 machine DESKTOP-06D7UBO
In computer 3 the value of y is 30
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 0.3 – message1_integer

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano message2.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ message2.cpp -o message2
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./message2
Just a normal process From rank 0 machine DESKTOP-06D7UBO
SEnding message to computer 3 from computer 1
Just a normal process From rank 2 machine DESKTOP-06D7UBO
in computer 3 the value of y is printed
 0  10  20  30  40  50  60  70  80  90 lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 0.4 – message2

Exercise 2 – Parallel sum of 1 to 10,000,000

Results

The total is 50,000,005,000,000 for every process count, which matches the formula N(N+1)/2.

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano sum_mpi.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ sum_mpi.cpp -o sum_mpi
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./sum_mpi
Number of processes: 4
Numbers: 1 to 10000000
Total sum: 50000005000000
Execution time: 0.006637 seconds
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 1 ./sum_mpi
Number of processes: 1
Numbers: 1 to 10000000
Total sum: 50000005000000
Execution time: 0.022621 seconds
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 2 ./sum_mpi
Number of processes: 2
Numbers: 1 to 10000000
Total sum: 50000005000000
Execution time: 0.011450 seconds
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./sum_mpi
Number of processes: 4
Numbers: 1 to 10000000
Total sum: 50000005000000
Execution time: 0.006683 seconds
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 2.1 – sum_mpi with 1, 2 and 4 processes

Processes	Time (s)	Speedup	Efficiency
1	0.022621	1.00	1.00
2	0.011450	1.98	0.99
4	0.006637	3.41	0.85

Exercise 3 – Monte Carlo estimation of Pi (10,000,000 trials)

Results

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano pi_monte_carlo.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ pi_monte_carlo.cpp -o pi_monte_carlo
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 1 ./pi_monte_carlo
Number of processes: 1
Total trials: 10000000
Points inside circle: 7855896
Estimated Pi: 3.1423584000
Actual Pi: 3.1415926536
Execution time: 2.654861 seconds
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 2 ./pi_monte_carlo
Number of processes: 2
Total trials: 10000000
Points inside circle: 7853689
Estimated Pi: 3.1414756000
Actual Pi: 3.1415926536
Execution time: 1.415602 seconds
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./pi_monte_carlo
Number of processes: 4
Total trials: 10000000
Points inside circle: 7853567
Estimated Pi: 3.1414268000
Actual Pi: 3.1415926536
Execution time: 0.970437 seconds
```

Figure 3.1 – pi_monte_carlo with 1, 2 and 4 processes

Processes	Points inside	Estimated Pi	Time (s)	Speedup
1	7,855,896	3.1423584	2.654861	1.00
2	7,853,689	3.1414756	1.415602	1.88
4	7,853,567	3.1414268	0.970437	2.74

The estimate is within about 0.001 of 3.14159 in every run. It changes slightly between runs because the random numbers differ; the error shrinks only with the square root of the number of trials.

Exercise 4 – Time and speedup graphs

Processors	Sum time (s)	Sum speedup	Pi time (s)	Pi speedup
1	0.022621	1.00	2.654861	1.00
2	0.011450	1.98	1.415602	1.88
4	0.006637	3.41	0.970437	2.74

1. Time vs number of processors

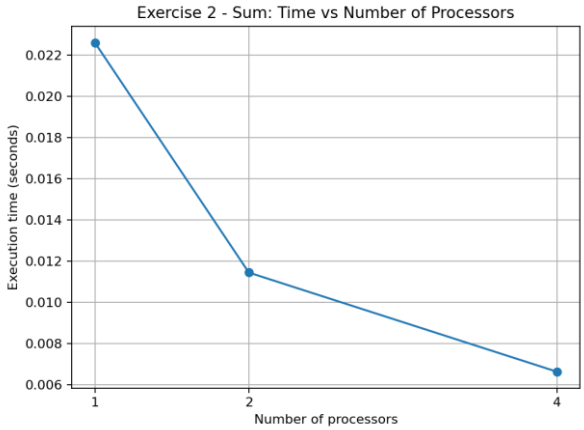


Figure 4.1 – Sum: time vs processors

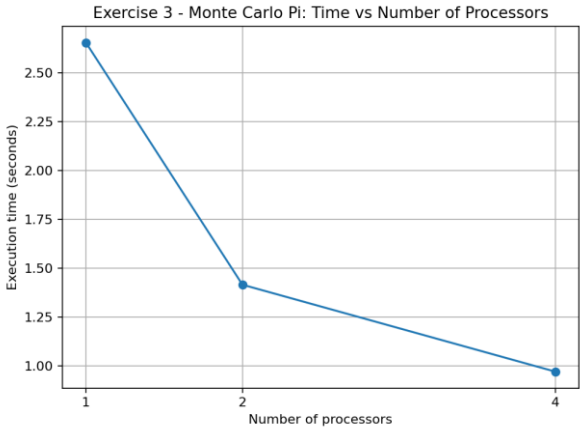


Figure 4.2 – Pi: time vs processors

2. Speedup

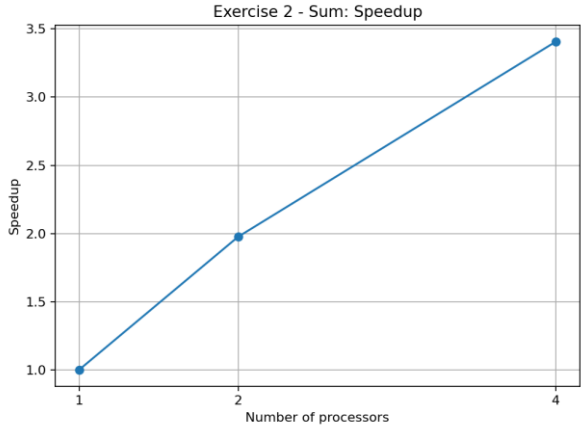


Figure 4.3 – Sum: speedup

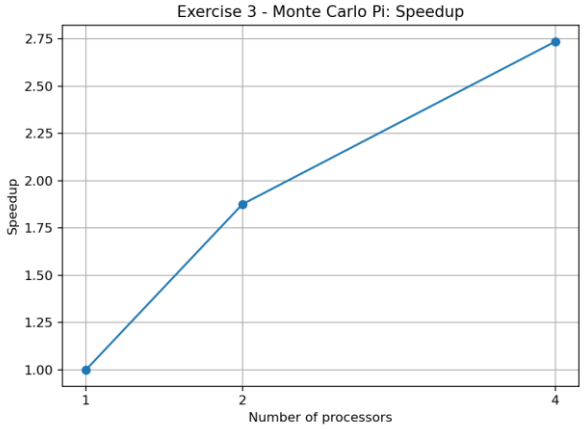


Figure 4.4 – Pi: speedup

Speedup is S(p) = T(1) / T(p).

Discussion

- Both programs get faster as processors are added, so the work parallelises well: each process works on an independent block and communication is only a single reduction.
- Speedup is below the ideal (S = p). For Pi it is 1.88 on 2 and 2.74 on 4 processes (efficiency 0.94 and 0.68). Process start-up, the reduction, the barrier and the cores/threads available on the machine all limit scaling, and running more processes than physical cores (--oversubscribe) makes this worse.
- The sum program takes only a few milliseconds, so timer resolution and operating-system noise have a larger effect on its speedup figures. Its near-linear result at 2 processes should be read with that in mind; a larger N would give more reliable numbers.

Exercise 5 – Mismatched source/destination and buffered send

5.1 Source and destination do not match

What happens: the program hangs and never finishes; it had to be stopped with Ctrl+C.

- Rank 3 waits in MPI_Recv for a message from rank 2. Rank 2 never sends anything, so the receive never completes.
- Rank 1's message is addressed to rank 3 but does not match the receive (wrong source), so it is never consumed. MPI_Ssend is synchronous: it only completes once the matching receive has started, so rank 1 also blocks. This is a deadlock.
- The line "in computer 3 the value of y is printed" never appears.

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ nano message2_mismatch.cpp
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpic++ message2_mismatch.cpp -o message2_mismatch
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./message2_mismatch
Just a normal process From rank 0 machine DESKTOP-06D7UBO
SEnding message to computer 3 from computer 1
Just a normal process From rank 2 machine DESKTOP-06D7UBO
^Clala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 5.1 – mismatched source: program hangs until interrupted (^C)

5.2 message2.cpp rewritten with MPI_Bsend

```
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$ mpirun --oversubscribe -np 4 ./message2_bsend
Just a normal process from rank 0 machine DESKTOP-06D7UBO
Sending buffered message from rank 1 to rank 3
In rank 3, the values of y are:
 0  10  20  30  40  50  60  70  80  90
Just a normal process from rank 2 machine DESKTOP-06D7UBO
lala@DESKTOP-06D7UBO:/mnt/c/Users/DELL/Desktop/SE3082-Lab8$
```

Figure 5.2 – message2_bsend: rank 3 receives 0 10 … 90

In message2_bsend.cpp, rank 1 allocates a buffer of 10 × sizeof(int) + MPI_BSEND_OVERHEAD bytes and attaches it with MPI_Buffer_attach. MPI_Bsend then copies the message into this buffer and returns immediately, without waiting for rank 3 to post its receive. The send array x is never modified after the call, and rank 3 receives into a separate array y, so no variable used for sending is replaced. MPI_Buffer_detach waits until the buffered message has been delivered, after which the buffer is freed. Rank 3 received 0 10 20 … 90, the same result as the original MPI_Ssend version.

Exercise 6 – Pi with MPI_ANY_SOURCE

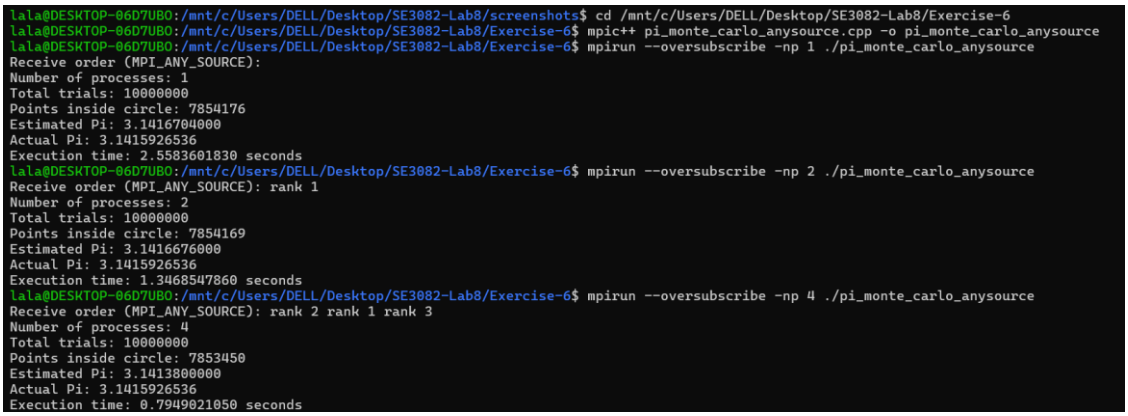


Figure 6.1 – pi_monte_carlo_anysource with 1, 2 and 4 processes

Comparison with the original Exercise 3

Processes	Ex 3 Pi	Ex 6 Pi	Ex 3 time (s)	Ex 6 time (s)	Same answer?
1	3.1423584	3.1416704	2.654861	2.558360	Close
2	3.1414756	3.1416676	1.415602	1.346855	Close
4	3.1414268	3.1413800	0.970437	0.794902	Close

- The computed value of Pi is the same to within random error. The counts differ slightly because Exercise 6 uses a different random sequence (fixed seed per rank) – not because of MPI_ANY_SOURCE.
- Execution times are very similar. The only communication is one small message per worker, so receiving with a wildcard instead of a reduction has no visible cost; the small time differences are run-to-run variation.
- With MPI_ANY_SOURCE the order in which rank 0 receives results is not fixed. In the 4-process run the order was rank 2, rank 1, rank 3. With a fixed source the order would always be 1, 2, 3 and rank 0 could be left waiting for a slow rank 1 while rank 2 had already finished. The final sum is unaffected because addition is commutative.

Exercise 7 – Pi with MPI_ANY_SOURCE and MPI_Bsend

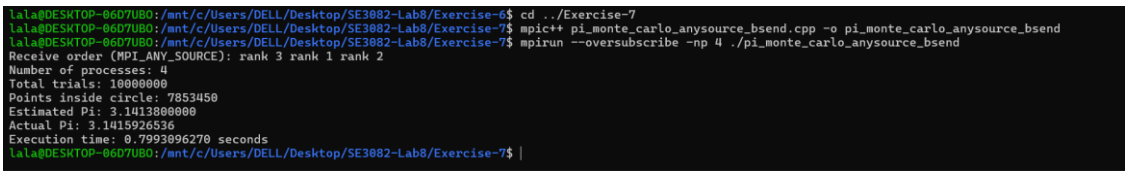


Figure 7.1 – pi_monte_carlo_anysource_bsend with 4 processes

Exercise 6 was changed so each worker sends its count to rank 0 with MPI_Bsend (using an attached buffer) instead of MPI_Send. Rank 0 still receives with MPI_ANY_SOURCE.

With 4 processes, the result was identical to Exercise 6 (7,853,450 points inside, Pi = 3.1413800), and the time was almost the same (0.799 s vs 0.795 s). Buffered sending does

not change the answer, and with one small message per worker it adds no visible cost. The receive order differed (rank 3, 1, 2 instead of 2, 1, 3), since it depends on which worker finishes first.